# Database Design

## CS411 - Team048 - TeamBuster

Josh Loftus, Yichang Zou, August Steffen, Boston Cox

## Connecting to GCP:

```
bostoff@cloudshell:~ (cs411-teambuster)$ gcloud sql connect cs411-teambuster --user=root --quiet
Starting Cloud SQL Proxy: [/usr/bin/cloud-sql-proxy cs411-teambuster:us-east1:cs411-teambuster --port 9470]
2026/10/21 19:28:24 Authorizing with Application Default Credentials
2026/10/21 19:28:24 [cs411-teambuster:us-east1:cs411-teambuster] Listening on 127.0.0.1:9470
2026/10/21 19:28:24 The proxy has started successfully and is ready for new connections!
Connecting to database with SQL user [root].Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 360335
Server version: 8.0.46-google (Google)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

## Data Definition Language:

```
CREATE TABLE Organization (
    Code        VARCHAR(8)   NOT NULL,
    UCode       VARCHAR(8),
    Name        VARCHAR(80),
    ShortName   VARCHAR(20),
    Class       CHAR(1),
    Parent      VARCHAR(8),
    StateCode   VARCHAR(8),
    Latitude    DECIMAL(9,4),
    Longitude   DECIMAL(9,4),
    PRIMARY KEY (Code),
    FOREIGN KEY (Parent)    REFERENCES Organization(Code),
    FOREIGN KEY (StateCode) REFERENCES Organization(Code)
);
```

```
CREATE TABLE LaunchSite (
    SiteCode   VARCHAR(8)   NOT NULL,
```

```
    Type        CHAR(2),
    Name        VARCHAR(80),
    StateCode   VARCHAR(8),
    Parent      VARCHAR(8),
    Latitude    DECIMAL(9,4),
    Longitude   DECIMAL(9,4),
    PRIMARY KEY (SiteCode),
    FOREIGN KEY (StateCode) REFERENCES Organization(Code),
    FOREIGN KEY (Parent)    REFERENCES Organization(Code)
);
```

```
CREATE TABLE Launch (
    LaunchTag       VARCHAR(12)  NOT NULL,
    Launch_Date     DATETIME,
    LV_Type         VARCHAR(30),
    Launch_Site     VARCHAR(8),
    Launch_Agency   VARCHAR(8),
    LaunchCode      VARCHAR(4),
    PRIMARY KEY (LaunchTag),
    FOREIGN KEY (Launch_Site)   REFERENCES LaunchSite(SiteCode),
    FOREIGN KEY (Launch_Agency) REFERENCES Organization(Code)
);
```

```
CREATE TABLE SpaceObject (
    JCAT           VARCHAR(12)  NOT NULL,
    Name           VARCHAR(30),
    Satcat         VARCHAR(8),
    LaunchTag      VARCHAR(12),
    Type           VARCHAR(12),
    Parent         VARCHAR(12),
    Owner          VARCHAR(8),
    State          VARCHAR(8),
    Manufacturer   VARCHAR(8),
    DryMass        DECIMAL(10,1),
    Status         VARCHAR(8),
    DDate          DATETIME,
    Perigee        INT,
    Apogee         INT,
```

```
    Inc           DECIMAL(5,2),
    OpOrbit       VARCHAR(8),
    PRIMARY KEY (JCAT),
    FOREIGN KEY (LaunchTag)    REFERENCES Launch(LaunchTag),
    FOREIGN KEY (Parent)       REFERENCES SpaceObject(JCAT),
    FOREIGN KEY (Owner)        REFERENCES Organization(Code),
    FOREIGN KEY (State)        REFERENCES Organization(Code),
    FOREIGN KEY (Manufacturer) REFERENCES Organization(Code)
);
```

```
CREATE TABLE User (
    UserID    INT           NOT NULL,
    Username  VARCHAR(30),
    Email     VARCHAR(100),
    Password  VARCHAR(255),
    PRIMARY KEY (UserID)
);



CREATE TABLE Watchlist (
    UserID   INT           NOT NULL,
    JCAT     VARCHAR(12)   NOT NULL,
    AddedAt  DATETIME,
    Note     VARCHAR(255),
    PRIMARY KEY (UserID, JCAT),
    FOREIGN KEY (UserID) REFERENCES `User`(UserID),
    FOREIGN KEY (JCAT)   REFERENCES SpaceObject(JCAT)
);
```

## Select Count(*) For Each Table:

```
mysql> select count(*) from Launch;
+----------+
| count(*) |
+----------+
|    76003 |
+----------+
1 row in set (0.19 sec)
```

```
mysql> select count(*) from Organization
    -> ;
+----------+
| count(*) |
+----------+
|     4141 |
+----------+
1 row in set (0.07 sec)
```

```
mysql> select count(*) from SpaceObject;
+----------+
| count(*) |
+----------+
|    69999 |
+----------+
1 row in set (0.62 sec)
```

```
mysql> select count(*) from User;
+----------+
| count(*) |
+----------+
|      105 |
+----------+
1 row in set (0.06 sec)
```

```
mysql> select count(*) from Watchlist;
+----------+
| count(*) |
+----------+
|     1000 |
+----------+
1 row in set (0.03 sec)
```

```
mysql> select count(*) from LaunchSite;
+----------+
| count(*) |
+----------+
|      712 |
+----------+
1 row in set (0.12 sec)
```

# Advanced Queries:

The below screenshots use “LIMIT 15” at the end to keep the output from being too large

1. Returns the number of objects owned by each organization and the organization’s location:

```
SELECT Org.Name, Org.Latitude, Org.Longitude, COUNT(JCAT) AS numObjects
FROM SpaceObject SP
JOIN Organization Org ON (SP.Owner = Org.Code)
GROUP BY Org.Code
ORDER BY numObjects DESC, Org.Name DESC;
```

```
+-----------------------------------------------------------------+----------+-----------+------------+
| Name                                                            | Latitude | Longitude | numObjects |
+-----------------------------------------------------------------+----------+-----------+------------+
| SpaceX (Seattle)                                                |  47.6700 | -122.1200 |      12805 |
| Zhongguo hangtian keji jituangongsi                             |  39.9200 |  116.3800 |       4396 |
| Rakentiye Voiska Strategicheskogo Naznacheniye                  |  55.7500 |   37.6200 |       4067 |
| Kosmichesikiye voyska Rossii                                    |  56.1800 |   36.9800 |       3164 |
| NASA Goddard Space Flight Center                                |  38.9900 |  -76.8900 |       3145 |
| Guojia Hangtian Ju                                              |  39.9200 |  116.3800 |       2597 |
| Glavnoye Upravleniye Kosmicheskhikh Sredstv                     |  55.7500 |   37.6200 |       1914 |
| Korporatsii Roskosmos                                           |  55.7500 |   37.6200 |       1820 |
| Rakentiye Voiska Strategicheskogo Naznacheniye (Rossisk. Fed.)  |  55.7500 |   37.6200 |       1530 |
| Glavnoye Upravleniye Kosmicheskhikh Sredstv                     |  55.7500 |   37.6200 |       1487 |
| Upravleniye Nachalnika Kosmicheskikh Sredstv                    |  55.7500 |   37.6200 |       1336 |
| US Air Force Space Systems Division                             |  33.9200 | -118.4100 |       1289 |
| Proto-Vozdhusnoy Oborony                                        |  55.7500 |   37.6200 |       1163 |
| Arianespace, Inc.                                               |  48.6200 |    2.4300 |       1129 |
| Voenno-Morskoy Flot SSSR                                        |  55.7500 |   37.6200 |       1111 |
+-----------------------------------------------------------------+----------+-----------+------------+
15 rows in set (0.68 sec)
```

2. (Need generated user data) For each user, returns the space object they added most recently to their watchlist, in ascending order by user name and object name:

```sql
SELECT u.Username, s.Name
FROM User AS u
JOIN Watchlist AS w ON w.UserID = u.UserID
JOIN SpaceObject AS s ON s.JCAT = w.JCAT
WHERE w.AddedAt = (
    SELECT MAX(w2.AddedAt)
    FROM Watchlist AS w2
    WHERE w2.UserID = w.UserID
)
ORDER BY u.Username, s.Name;
```

```
+----------+--------------------------+
| Username | Name                     |
+----------+--------------------------+
| alice_12 | KRT-10                   |
| alice_24 | deb Fregat SBB           |
| alice_36 | deb Kosmos-397           |
| alice_48 | FEU-170 cover?           |
| alice_60 | deb 11S772               |
| alice_72 | Flock 3p-43              |
| alice_96 | EO-24 EVA-5 debris (MSB) |
| bob_103  | deb Molniya-3            |
| bob_19   | deb Ablestar 008         |
| bob_31   | deb Transtage 4          |
| bob_43   | deb Kosmos-777           |
| bob_55   | Progress M-14            |
| bob_67   | Starlink 5563            |
| bob_7    | Garbage bag              |
| bob_79   | deb Kosmos-57            |
+----------+--------------------------+
15 rows in set (0.05 sec)
```

3. Returns the launch sites that have been used the most in 2026 (hosted the most launches)

```sql
SELECT LS.Name, COUNT(L.LaunchTag) AS numLaunches
FROM LaunchSite LS
JOIN Launch L ON (LS.SiteCode = L.Launch_Site)
WHERE L.Launch_Date >= '2026-01-01'
GROUP BY LS.SiteCode
ORDER BY numLaunches DESC, LS.Name DESC;
```

```
+-------------------------------------------------------------------------------+-------------+
| Name                                                                          | numLaunches |
+-------------------------------------------------------------------------------+-------------+
| Mobile launchers in Iran?                                                     |         222 |
| South Base, Vandenberg Space Force Base, California                           |          64 |
| Cape Canaveral Air Station, Florida                                           |          56 |
| Jiuquan Space Center, Nei Monggol Zizhiqu, China                              |          27 |
| Rocket Lab Launch Complex 1, Onenui Station, Mahia Peninsula                  |          15 |
| Hainan Shangye Hangtian Fashe Chang (Hainan Commercial Space Launch Site)     |          14 |
| Taiyuan weixing fashe zhongxin                                                |          12 |
| Sunan Airport, Pyonyang                                                       |          10 |
| 1-y Gosudarstvenniy Ispitatelniy Kosmodrom MO RF                              |           8 |
| Wenchang Space Center, Hainan                                                 |           7 |
| Kalma Airport, Wonsan/Kittaeryong (Gitdaeryung), Kangwon-do                   |           7 |
| Xichang Space Center (Songlin), Sichuan, China                                |           6 |
| Centre Spatial Guyanais, Kourou, Guyane Francaise                             |           6 |
| Poker Flat Research Range, Ft Wainwright, Alaska                              |           5 |
| GIK-5, Baykonur, Kazakstan                                                    |           5 |
+-------------------------------------------------------------------------------+-------------+
15 rows in set (4.97 sec)
```

# Indexing:

# Query 1:

Command:

```
EXPLAIN ANALYZE
SELECT Org.Name, Org.Latitude, Org.Longitude, COUNT(JCAT) AS numObjects
FROM SpaceObject SP
JOIN Organization Org ON (SP.Owner = Org.Code)
GROUP BY Org.Code
ORDER BY numObjects DESC, Org.Name DESC;
```

Analysis Before Indexing:

```
| -> Sort: numObjects DESC, Org.`Name` DESC  (actual time=93.5..93.9 rows=1554 loops=1)
    -> Stream results  (cost=36457 rows=3892) (actual time=2.52..85.7 rows=1554 loops=1)
        -> Group aggregate: count(SP.JCAT)  (cost=36457 rows=3892) (actual time=2.51..84.4 rows=1554 loops=1)
            -> Nested loop inner join  (cost=21126 rows=153307) (actual time=2.47..69.8 rows=69999 loops=1)
                -> Index scan on Org using PRIMARY  (cost=413 rows=3892) (actual time=0.305..4.91 rows=4141 loops=1)
                -> Covering index lookup on SP using Owner (Owner=Org.`Code`)  (cost=1.38 rows=39.4) (actual time=0.00494..0.0137 rows=16.9 loops=4141)
 |
```

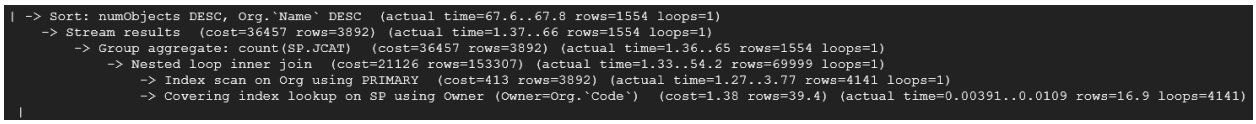
First Attempt:

```
| -> Sort: numObjects DESC, Org.`Name` DESC  (actual time=178..178 rows=1554 loops=1)
    -> Stream results  (cost=30347 rows=3892) (actual time=0.189..177 rows=1554 loops=1)
        -> Group aggregate: count(SP.JCAT)  (cost=30347 rows=3892) (actual time=0.186..176 rows=1554 loops=1)
            -> Nested loop inner join  (cost=17945 rows=124015) (actual time=0.161..164 rows=69999 loops=1)
                -> Index scan on Org using PRIMARY  (cost=413 rows=3892) (actual time=0.126..2.88 rows=4141 loops=1)
                -> Covering index lookup on SP using idx_owner (Owner=Org.`Code`)  (cost=1.32 rows=31.9) (actual time=0.0042..0.0376 rows=16.9 loops=4141)
 |
```

```
CREATE INDEX idx_owner ON SpaceObject (Owner);
```

Performance in estimated cost improved slightly after adding an index on Owner. The cost for the stream results and the group aggregate decreased (36457 -> 30347). Costs for the nested loop inner join and index lookup on SP also decreased (21126 -> 17945) & (1.38 -> 1.32) respectively.

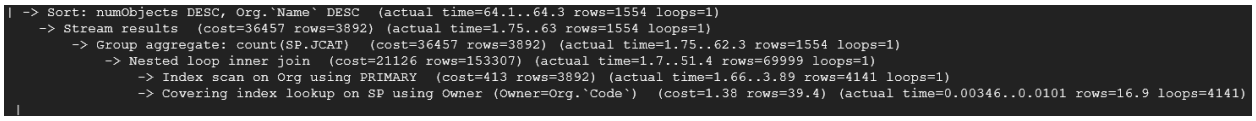
Second Attempt:




```
CREATE INDEX idx_org_name ON Organization (Name);
```

Our second attempt at indexing did not affect the query performance :(

Third Attempt:

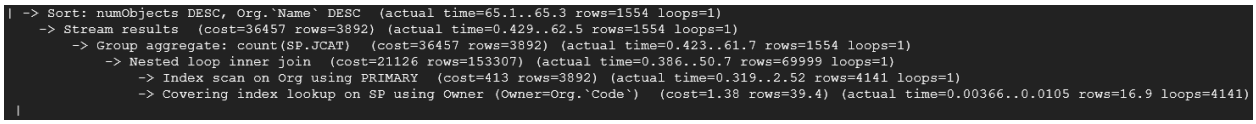



```
CREATE INDEX idx_latitude ON Organization (Latitude);
```

Our third attempt at indexing also did not affect the query performance :(

(An index on Organization.Longitude does not impact performance either)

Fourth Attempt:




```
CREATE INDEX idx_lat_long ON Organization (Latitude, Longitude);
```

The fourth attempt at indexing did not impact performance either :(

For the first query, the index on Organizations.Owner is the only index that actually improved the query performance, so that is the one that we would choose. We believe that the reason for this is because Organizations.Owner is used when joining Organizations to SpaceObject. We think the second index on Org.Name didn't help because the query is already sorting based on the aggregation, so it isn't directly helpful. We also think that the third index on Latitude and Longitude were unhelpful

because the SELECT clause happens last, after all of the heavy lifting has already been done.

# Query 2:

Command:

```
EXPLAIN ANALYZE
SELECT u.Username, s.Name
FROM User AS u
JOIN Watchlist AS w ON w.UserID = u.UserID
JOIN SpaceObject AS s ON s.JCAT = w.JCAT
WHERE w.AddedAt = (
    SELECT MAX(w2.AddedAt)
    FROM Watchlist AS w2
    WHERE w2.UserID = w.UserID
)
ORDER BY u.Username, s.Name;
```

Analysis Before Indexing:

```
| -> Sort: u.Username, s.`Name`  (actual time=19.6..19.6 rows=136 loops=1)
    -> Stream results  (cost=1205 rows=1050) (actual time=1.61..16 rows=136 loops=1)
        -> Nested loop inner join  (cost=1205 rows=1050) (actual time=1.6..15.9 rows=136 loops=1)
            -> Nested loop inner join  (cost=142 rows=1050) (actual time=1.57..9.87 rows=136 loops=1)
                -> Table scan on u  (cost=10.8 rows=105) (actual time=0.145..0.204 rows=105 loops=1)
                -> Filter: (w.AddedAt = (select #2))  (cost=0.262 rows=10) (actual time=0.0524..0.0916 rows=1.3 loops=105)
                    -> Index lookup on w using PRIMARY (UserID=u.UserID)  (cost=0.262 rows=10) (actual time=0.00799..0.00983 rows=9.52 loops=105)
                    -> Select #2 (subquery in condition; dependent)
                        -> Aggregate: max(w2.AddedAt)  (cost=2.25 rows=1) (actual time=0.00742..0.00745 rows=1 loops=950)
                            -> Index lookup on w2 using PRIMARY (UserID=w.UserID)  (cost=1.25 rows=10) (actual time=0.00383..0.00528 rows=10 loops=950)
            -> Single-row index lookup on s using PRIMARY (JCAT=w.JCAT)  (cost=0.912 rows=1) (actual time=0.0441..0.0441 rows=1 loops=136)
 |
```

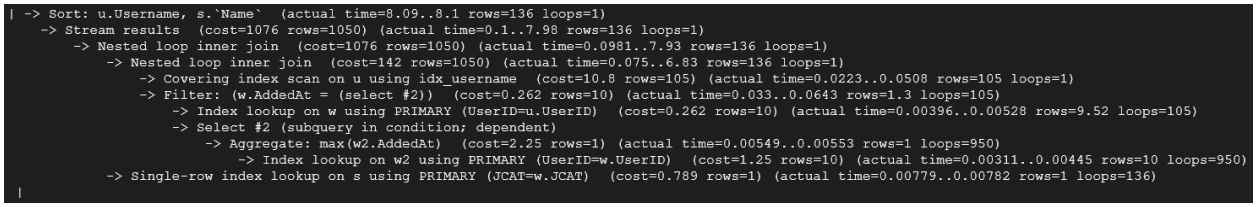
First Attempt:

```
| -> Sort: u.Username, s.`Name`  (actual time=81.2..81.2 rows=136 loops=1)
    -> Stream results  (cost=1064 rows=1050) (actual time=3.61..80.1 rows=136 loops=1)
        -> Nested loop inner join  (cost=1064 rows=1050) (actual time=3.61..79.9 rows=136 loops=1)
            -> Nested loop inner join  (cost=142 rows=1050) (actual time=3.56..11.2 rows=136 loops=1)
                -> Table scan on u  (cost=10.8 rows=105) (actual time=0.0687..0.128 rows=105 loops=1)
                -> Filter: (w.AddedAt = (select #2))  (cost=0.262 rows=10) (actual time=0.068..0.105 rows=1.3 loops=105)
                    -> Index lookup on w using PRIMARY (UserID=u.UserID)  (cost=0.262 rows=10) (actual time=0.00558..0.0072 rows=9.52 loops=105)
                    -> Select #2 (subquery in condition; dependent)
                        -> Aggregate: max(w2.AddedAt)  (cost=2.25 rows=1) (actual time=0.00711..0.00714 rows=1 loops=950)
                            -> Index lookup on w2 using PRIMARY (UserID=w.UserID)  (cost=1.25 rows=10) (actual time=0.00352..0.00491 rows=10 loops=950)
            -> Single-row index lookup on s using PRIMARY (JCAT=w.JCAT)  (cost=0.778 rows=1) (actual time=0.505..0.505 rows=1 loops=136)
 |
```

```
CREATE INDEX idx_addedat ON Watchlist (AddedAt);
```

Our first attempt at indexing decreased the cost of the stream results and the first nested loop inner join (1205 -> 1064). The cost of the single-row index lookup on s also dropped from 0.912 -> 0.778.
This is because AddedAt is involved in the MAX operation, so the index reduces its cost.

Second Attempt:

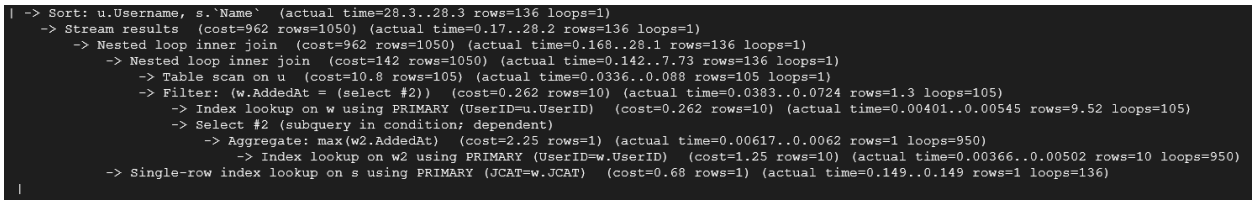


```
| -> Sort: u.Username, s.`Name`  (actual time=8.09..8.1 rows=136 loops=1)
    -> Stream results  (cost=1076 rows=1050) (actual time=0.1..7.98 rows=136 loops=1)
        -> Nested loop inner join  (cost=1076 rows=1050) (actual time=0.0981..7.93 rows=136 loops=1)
            -> Nested loop inner join  (cost=142 rows=1050) (actual time=0.075..6.83 rows=136 loops=1)
                -> Covering index scan on u using idx_username  (cost=10.8 rows=105) (actual time=0.0223..0.0508 rows=105 loops=1)
                -> Filter: (w.AddedAt = (select #2))  (cost=0.262 rows=10) (actual time=0.033..0.0643 rows=1.3 loops=105)
                    -> Index lookup on w using PRIMARY (UserID=u.UserID)  (cost=0.262 rows=10) (actual time=0.00396..0.00528 rows=9.52 loops=105)
                    -> Select #2 (subquery in condition; dependent)
                        -> Aggregate: max(w2.AddedAt)  (cost=2.25 rows=1) (actual time=0.00549..0.00553 rows=1 loops=950)
                            -> Index lookup on w2 using PRIMARY (UserID=w.UserID)  (cost=1.25 rows=10) (actual time=0.00311..0.00445 rows=10 loops=950)
            -> Single-row index lookup on s using PRIMARY (JCAT=w.JCAT)  (cost=0.789 rows=1) (actual time=0.00779..0.00782 rows=1 loops=136)
 |
```

```
CREATE INDEX idx_username ON User (Username);
```

Our second attempt at indexing decreased the cost of the stream results and first nested loop inner join from 1205 -> 1076 (not quite as good as the first attempt). The cost for the single-row index lookup on s also decreased from 0.912 -> 0.789 (also not as good as the first attempt).
This index helps because it is involved in the ordering of the results, but is not as good because it's not involved in the main part of the query.

Third Attempt:

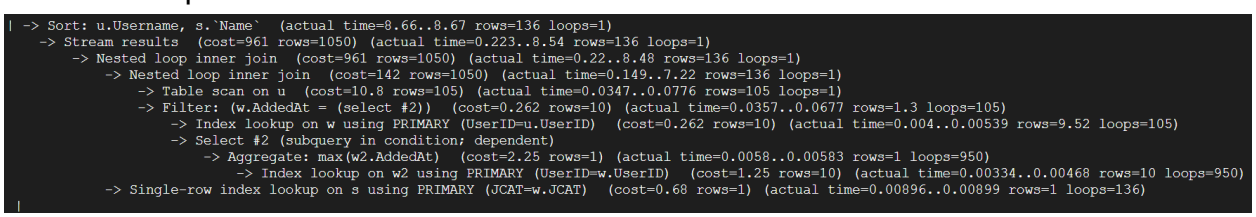


```
| -> Sort: u.Username, s.`Name`  (actual time=28.3..28.3 rows=136 loops=1)
    -> Stream results  (cost=962 rows=1050) (actual time=0.17..28.2 rows=136 loops=1)
        -> Nested loop inner join  (cost=962 rows=1050) (actual time=0.168..28.1 rows=136 loops=1)
            -> Nested loop inner join  (cost=142 rows=1050) (actual time=0.142..7.73 rows=136 loops=1)
                -> Table scan on u  (cost=10.8 rows=105) (actual time=0.0336..0.088 rows=105 loops=1)
                -> Filter: (w.AddedAt = (select #2))  (cost=0.262 rows=10) (actual time=0.0383..0.0724 rows=1.3 loops=105)
                    -> Index lookup on w using PRIMARY (UserID=u.UserID)  (cost=0.262 rows=10) (actual time=0.00401..0.00545 rows=9.52 loops=105)
                    -> Select #2 (subquery in condition; dependent)
                        -> Aggregate: max(w2.AddedAt)  (cost=2.25 rows=1) (actual time=0.00617..0.0062 rows=1 loops=950)
                            -> Index lookup on w2 using PRIMARY (UserID=w.UserID)  (cost=1.25 rows=10) (actual time=0.00366..0.00502 rows=10 loops=950)
            -> Single-row index lookup on s using PRIMARY (JCAT=w.JCAT)  (cost=0.68 rows=1) (actual time=0.149..0.149 rows=1 loops=136)
 |
```

```
CREATE INDEX idx_name ON SpaceObject (Name);
```

The third attempt at indexing decreased the cost for the stream results and first nested loop inner join from 1205 -> 962. The cost of the single-row index lookup on s also decreased (0.912 -> 0.680).
This one is interesting. While SpaceObject.Name is only involved in the ORDER BY and SELECT clauses, it has great improvements in performance. It is likely that the query optimizer was able to find a better optimization with the existence of this index compared to the other indexes.

Fourth Attempt:



```
| -> Sort: u.Username, s.`Name`  (actual time=8.66..8.67 rows=136 loops=1)
    -> Stream results  (cost=961 rows=1050) (actual time=0.223..8.54 rows=136 loops=1)
        -> Nested loop inner join  (cost=961 rows=1050) (actual time=0.22..8.48 rows=136 loops=1)
            -> Nested loop inner join  (cost=142 rows=1050) (actual time=0.149..7.22 rows=136 loops=1)
                -> Table scan on u  (cost=10.8 rows=105) (actual time=0.0347..0.0776 rows=105 loops=1)
                -> Filter: (w.AddedAt = (select #2))  (cost=0.262 rows=10) (actual time=0.0357..0.0677 rows=1.3 loops=105)
                    -> Index lookup on w using PRIMARY (UserID=u.UserID)  (cost=0.262 rows=10) (actual time=0.004..0.00539 rows=9.52 loops=105)
                    -> Select #2 (subquery in condition; dependent)
                        -> Aggregate: max(w2.AddedAt)  (cost=2.25 rows=1) (actual time=0.0058..0.00583 rows=1 loops=950)
                            -> Index lookup on w2 using PRIMARY (UserID=w.UserID)  (cost=1.25 rows=10) (actual time=0.00334..0.00468 rows=10 loops=950)
            -> Single-row index lookup on s using PRIMARY (JCAT=w.JCAT)  (cost=0.68 rows=1) (actual time=0.00896..0.00899 rows=1 loops=136)
 |
```

```
CREATE INDEX idx_user_addedat ON Watchlist (UserID, AddedAt);
```

Our fourth attempt at indexing has a performance close to the third attempt. It decreased the cost for the stream results and first nested loop inner join from 1205 -> 961. The single-row index on s also decreased from 0.912 to 0.680.
This is likely the best index of the bunch. The combination of UserID and AddedAt for Watchlist allows the subquery to run much more efficiently as it is better able to determine the max AddedAt for a specific UserID.

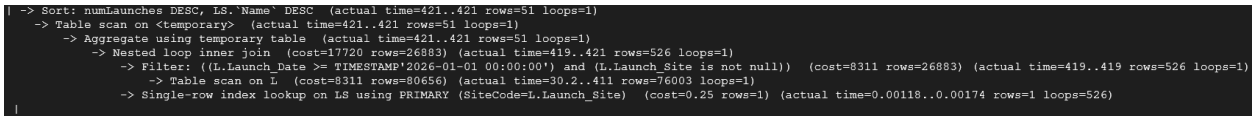
The indexing scheme that we would choose for the second query would be either the third or fourth attempt, since the cost for stream results and the first nested loop inner join differ by only 1.

# Query 3:

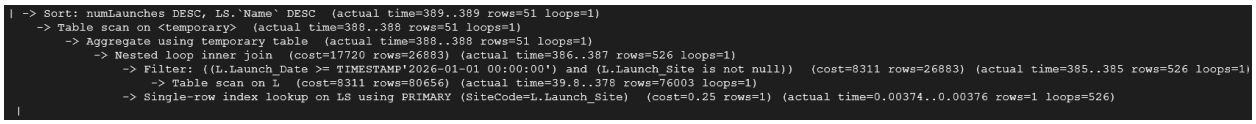
Command:

```
EXPLAIN ANALYZE
SELECT LS.Name, COUNT(L.LaunchTag) AS numLaunches
FROM LaunchSite LS
JOIN Launch L ON (LS.SiteCode = L.Launch_Site)
WHERE L.Launch_Date >= '2026-01-01'
GROUP BY LS.SiteCode
ORDER BY numLaunches DESC, LS.Name DESC;
```

Analysis Before Indexing:



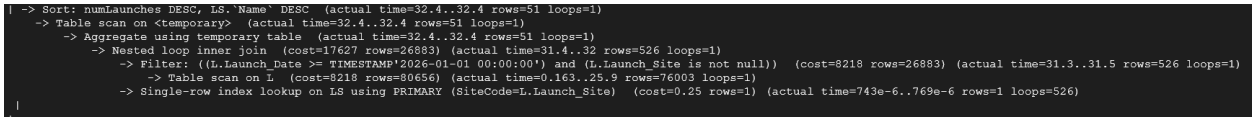

First Attempt:




```
CREATE INDEX idx_name ON LaunchSite (Name);
```

Our first attempt at indexing did not affect the performance of the query :(
This is likely because LaunchSite.Name is only involved in the select clause, and is the second argument of the order by clause.

Second Attempt:

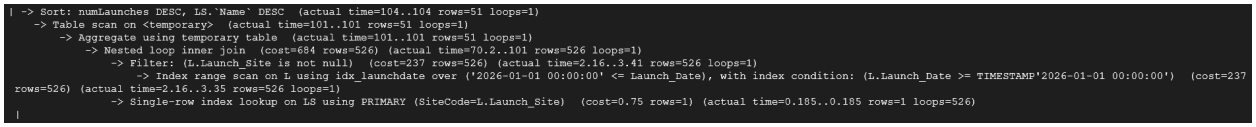


```
| -> Sort: numLaunches DESC, LS.`Name` DESC  (actual time=32.4..32.4 rows=51 loops=1)
    -> Table scan on <temporary>  (actual time=32.4..32.4 rows=51 loops=1)
        -> Aggregate using temporary table  (actual time=32.4..32.4 rows=51 loops=1)
            -> Nested loop inner join  (cost=17627 rows=26883) (actual time=31.4..32 rows=526 loops=1)
                -> Filter: ((L.Launch_Date >= TIMESTAMP'2026-01-01 00:00:00') and (L.Launch_Site is not null))  (cost=8218 rows=26883) (actual time=31.3..31.5 rows=526 loops=1)
                    -> Table scan on L  (cost=8218 rows=80656) (actual time=0.163..25.9 rows=76003 loops=1)
                -> Single-row index lookup on LS using PRIMARY (SiteCode=L.Launch_Site)  (cost=0.25 rows=1) (actual time=743e-6..769e-6 rows=1 loops=526)
|
```

```
CREATE INDEX idx_launchsite ON Launch (Launch_Site);
```

Our second attempt at indexing slightly reduced the cost of the nested loop inner join (17720 -> 17627) and the filter and table scan on Launch (8311 -> 8218).
The improvement likely comes from the fact that Launch.Launch_Site is involved in the joining of the two tables. The fact that the improvement is small suggests that the join is not the bottleneck of the query.

Third Attempt:



```
| -> Sort: numLaunches DESC, LS.`Name` DESC  (actual time=104..104 rows=51 loops=1)
    -> Table scan on <temporary>  (actual time=101..101 rows=51 loops=1)
        -> Aggregate using temporary table  (actual time=101..101 rows=51 loops=1)
            -> Nested loop inner join  (cost=684 rows=526) (actual time=70.2..101 rows=526 loops=1)
                -> Filter: (L.Launch_Site is not null)  (cost=237 rows=526) (actual time=2.16..3.41 rows=526 loops=1)
                    -> Index range scan on L using idx_launchdate over ('2026-01-01 00:00:00' <= Launch_Date), with index condition: (L.Launch_Date >= TIMESTAMP'2026-01-01 00:00:00')  (cost=237
rows=526) (actual time=2.16..3.35 rows=526 loops=1)
                -> Single-row index lookup on LS using PRIMARY (SiteCode=L.Launch_Site)  (cost=0.75 rows=1) (actual time=0.185..0.185 rows=1 loops=526)
|
```

```
CREATE INDEX idx_launchdate ON Launch (Launch_Date);
```

Our third attempt at indexing provided a HUGE performance improvement. The cost for the nested loop inner join, filter, table scan, dropped from (17720 -> 684), (8311 -> 237), and (8311 -> 237), respectively. However, the cost for the single-row index lookup on LaunchSite increased from (0.25 -> 0.75).
Not only does the index make executing the where clause much more efficient, eliminating most of the rows from Launch before doing anything else will cause those improvements to cascade through the rest of the query.

The indexing scheme we should go with is definitely the third one since it provided massive improvements in the performance of the query compared to the other two.